



UNIVERSITY OF CRAIOVA  
FACULTY OF AUTOMATION, COMPUTERS AND ELECTRONICS  
DEPARTMENT OF COMPUTERS AND INFORMATION  
TECHNOLOGY  
PROGRAM OF ARTIFICIAL INTELLIGENCE AND APPLIED  
COMPUTING



## WATER QUALITY MONITORING NETWORK

POPA GEORGIANA DANIELA

SOLARI ROBERTA

COANDĂ AMELIA

SCIENTIFIC COORDINATOR

MURĂREȚU IONUȚ

January 2026

CRAIOVA

# SUMMARY

|             |                                                 |          |
|-------------|-------------------------------------------------|----------|
| <b>1.</b>   | <b>INTRODUCTION .....</b>                       | <b>4</b> |
| <b>1.1.</b> | <b>PROBLEM DEFINITION.....</b>                  | <b>4</b> |
| <b>1.2.</b> | <b>SIGNIFICANCE.....</b>                        | <b>4</b> |
| <b>1.3.</b> | <b>JUSTIFICATION OF APPROACH .....</b>          | <b>4</b> |
| <b>2.</b>   | <b>EDGE INTELLIGENCE METHODOLOGY .....</b>      | <b>4</b> |
| <b>2.1.</b> | <b>METHOD DESCRIPTION .....</b>                 | <b>4</b> |
| <b>2.2.</b> | <b>MODEL ARCHITECTURE.....</b>                  | <b>4</b> |
| <b>3.</b>   | <b>SOFTWARE DESIGN AND IMPLEMENTATION .....</b> | <b>5</b> |
| <b>3.1.</b> | <b>ARCHITECTURE OVERVIEW .....</b>              | <b>5</b> |
| <b>3.2.</b> | <b>MODULES AND FUNCTIONS .....</b>              | <b>6</b> |
| <b>3.3.</b> | <b>LIBRARIES USED .....</b>                     | <b>6</b> |
| <b>4.</b>   | <b>DATASET .....</b>                            | <b>6</b> |
| <b>4.1.</b> | <b>DESCRIPTION .....</b>                        | <b>6</b> |
| <b>4.2.</b> | <b>GENERATION METHOD.....</b>                   | <b>6</b> |
| <b>5.</b>   | <b>EXPERIMENTS AND RESULTS.....</b>             | <b>7</b> |
| <b>5.1.</b> | <b>EXPERIMENTAL SETUP .....</b>                 | <b>7</b> |
| <b>5.2.</b> | <b>RESULTS ANALYSIS .....</b>                   | <b>7</b> |
| <b>6.</b>   | <b>CONCLUSION.....</b>                          | <b>7</b> |
| <b>6.1.</b> | <b>SUMMARY OF FINDINGS.....</b>                 | <b>7</b> |
| <b>6.2.</b> | <b>LIMITATIONS .....</b>                        | <b>7</b> |
| <b>6.3.</b> | <b>FUTURE WORK .....</b>                        | <b>8</b> |
|             | <b>REFERENCES.....</b>                          | <b>9</b> |

# LIST OF FIGURES

|                                  |   |
|----------------------------------|---|
| FIGURE 1. IOT ARCHITECTURE ..... | 5 |
|----------------------------------|---|

# 1. INTRODUCTION

## 1.1. Problem Definition

Water pollution is a major environmental challenge. Traditional monitoring relies on manual sampling, which is slow and reactive. This project addresses the need for a real-time system capable of monitoring water parameters (pH, Turbidity, Temperature) and detecting anomalies immediately.

## 1.2. Significance

- **Environmental Protection:** Early detection of contaminants saves aquatic ecosystems.
- **Smart Cities:** Integrating environmental sensing into automated networks.
- **Resource Constraints:** Real-world sensors often run on batteries with weak processors, making heavy algorithms impractical.

## 1.3. Justification of Approach

While deep learning models are powerful, they are often too heavy for simple edge devices. This project utilizes an **Adaptive Threshold Model** (a statistical approach to Edge AI). This approach is chosen because it handles "data drift" (natural changes in water) and noise effectively while remaining lightweight enough to run on microcontrollers like the ESP32.

# 2. EDGE INTELLIGENCE METHODOLOGY

## 2.1. Method Description

The project employs a **Statistical Moving Window Model** for anomaly detection. Unlike static rules (e.g., "pH must always be 7"), this model learns from the immediate past to determine what is currently "normal."

## 2.2. Model Architecture

The intelligence is built on a sliding window algorithm:

1. **Input Layer:** Raw sensor data (Temperature, pH, Turbidity).
2. **Processing Layer (The "Window"):** A memory buffer stores the last  $N$  readings (Window Size).
3. **Adaptive Logic:**

- The system calculates the **Moving Average** of the history buffer.
- It defines a **Dynamic Safety Margin** around this average.
- Formula:  $\text{Threshold}_{\{\min, \max\}} = \text{Average} \pm \text{Margin}$ .

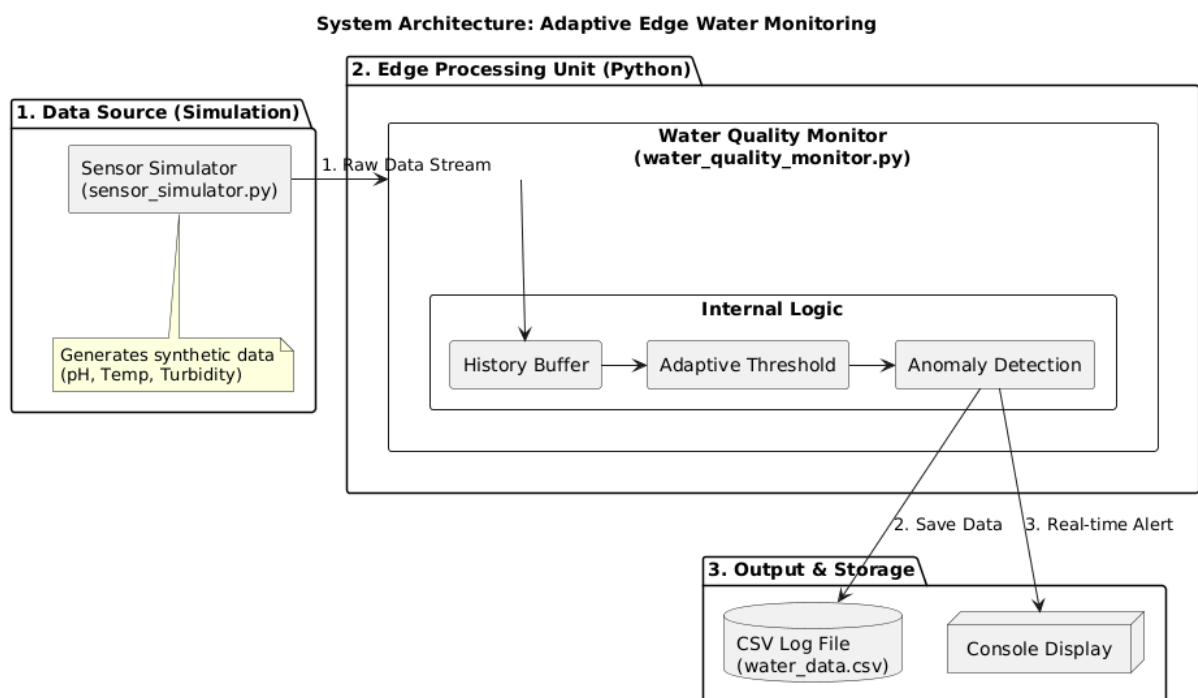
#### 4. Classification:

- If the current reading is inside the dynamic threshold → **Status: GOOD**.
- If the current reading is outside → **Status: BAD** (Anomaly Detected).

## 3. SOFTWARE DESIGN AND IMPLEMENTATION

### 3.1. Architecture Overview

The software is designed as a modular Python application simulating an IoT Edge Node.



**Figure 1. IoT Architecture**

### 3.2. Modules and Functions

- **sensor\_simulator.py**: This module acts as the hardware layer. It generates synthetic data for three distinct nodes using random uniform distributions to mimic sensor noise.
- **water\_quality\_monitor.py**: The main processing unit. It contains:
  - `read_sensors()`: Fetches data.
  - `adaptive_threshold()`: The core intelligence function that calculates dynamic limits based on history.
- **Main Loop**: Manages the data flow, logging, and console output.

### 3.3. Libraries Used

- `random`: For generating simulation data.
- `csv`: For persistent data storage.
- `collections (defaultdict)`: For managing the history buffers of multiple nodes efficiently.

## 4. DATASET

### 4.1. Description

Due to the difficulty of obtaining real-time labeled anomaly data for specific local water bodies, this project uses a Synthetic Dataset.

### 4.2. Generation Method

The dataset is generated dynamically at runtime by the `sensor_simulator.py` module. It simulates realistic ranges based on standard water quality parameters:

- **Temperature**: 8°C - 28°C
- **pH**: 6.0 - 9.0 (with simulated drift)
- **Turbidity**: 0 - 6 NTU

**Availability**: The data is generated live and stored in `water_data.csv` during execution.

## 5. EXPERIMENTS AND RESULTS

### 5.1. Experimental Setup

The experiment was conducted by running the software simulation continuously to observe the system's reaction to random data fluctuations.

**Hyperparameters:** To ensure reproducibility, the following configuration was used:

- **Window Size:** 10 samples (The memory span of the model).
- **Safety Margin:** 0.7 (The allowed deviation from the average pH).
- **Turbidity Hard Limit:** 5.0 (Absolute cutoff for clear water).

### 5.2. Results Analysis

The system successfully monitored 3 nodes simultaneously.

- **Drift Adaptation:** As the random pH values shifted slightly within normal bounds, the system adjusted the "safe range" (e.g., from 6.5-8.5 to 6.8-8.2) without triggering false alarms.
- **Anomaly Detection:** When the simulator produced a high turbidity value ( $>5$ ) or a pH spike outside the calculated window, the system correctly flagged the status as **"BAD"**.

## 6. CONCLUSION

### 6.1. Summary of Findings

This project successfully designed and implemented a real-time water quality monitoring system. The use of adaptive thresholding proved to be an effective "Edge Intelligence" technique, capable of distinguishing between natural data drift and actual pollution anomalies.

### 6.2. Limitations

- **Synthetic Data:** The system was tested on simulated data, which may not capture the complex chemical interactions of real water.
- **Hardware:** The experiment was software-only; real sensors might introduce electrical noise not accounted for here.

### **6.3.Future Work**

Future development will involve deploying this code onto physical **ESP32** microcontrollers and integrating a **TinyML** Deep Learning classifier (using TensorFlow Lite) to categorize specific types of pollutants (e.g., distinguishing oil from algae).



## REFERENCES

- [1] Pete Warden. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.
- [2] Python Software Foundation. *CSV File Reading and Writing*. <https://docs.python.org/3/library/csv.html>, 2024.
- [3] Real Python. *Python's Deque: Implement Efficient Queues and Stacks*. <https://realpython.com/python-deque/>, 2023.
- [4] Jason Brownlee. *Moving Average Smoothing for Time Series Anomaly Detection*. <https://machinelearningmastery.com/moving-average-smoothing-for-time-series-forecasting-python/>, 2021.
- [5] Espressif Systems. *ESP32 Technical Reference Manual*. <https://www.espressif.com/en/support/documents/technical-documents>, 2024.
- [6] GitHub Repository. *IoT Water Quality Monitor Code*. <https://github.com/GeorgianaPopa/Water-Quality-Monitoring-Network-.git>, 2025.